

Lecture 6: Continuous Random Variables, PDFs, CDFs

Jamie Haddock

Table of contents

0.1	Why do we need a new kind of random variable?	1
0.2	Example: the continuous uniform random variable	2
0.3	Example: a PDF can take arbitrarily large values	3
0.4	Expectation of a continuous random variable	3
0.5	The Exponential random variable	4
0.6	Cumulative Distribution Functions	6
0.7	Properties of the CDF	6
0.8	Geometric and Exponential CDFs	7

0.1 Why do we need a new kind of random variable?

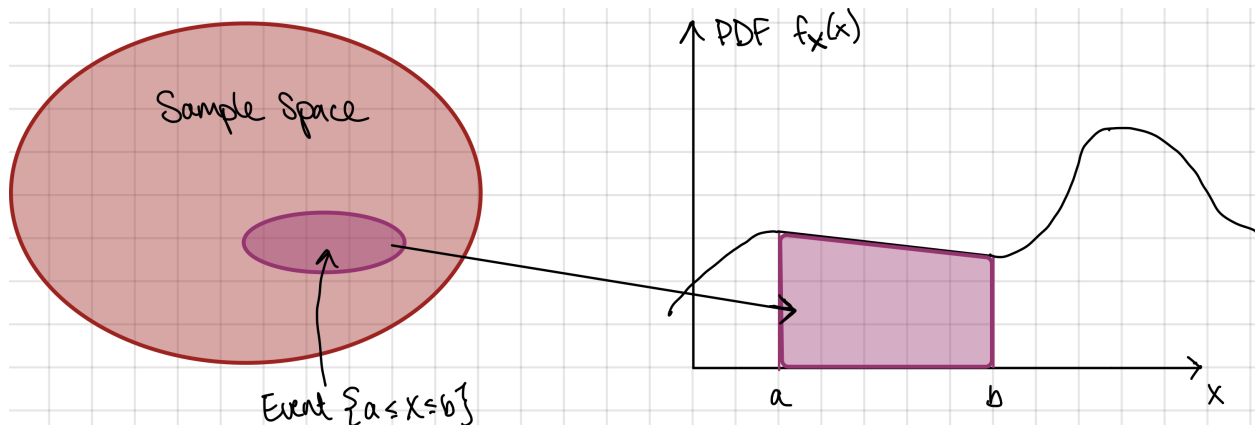
Every random variable so far has been **discrete**: a finite or countably infinite list of possible values, each with its own probability mass. But plenty of quantities we'd like to model — a waiting time, a position, a measurement error — can take *any* real value in an interval. There's no way to list them out one at a time.

i Definition: Continuous random variable, PDF

A random variable X is **continuous** if there is a function $f_X(x) \geq 0$, called its **probability density function (PDF)**, such that for every interval $[a, b]$,

$$P(a \leq X \leq b) = \int_a^b f_X(x) dx, \quad \int_{-\infty}^{\infty} f_X(x) dx = 1.$$

Instead of bars of probability sitting on individual points (a PMF), probability is now **area under a curve**.



0.2 Example: the continuous uniform random variable

i Definition: Uniform(a, b)

$$f_X(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise.} \end{cases}$$

Every value in $[a, b]$ is “equally likely,” in the sense that the density is flat — probability of a sub-interval depends only on its *length*, not its location.

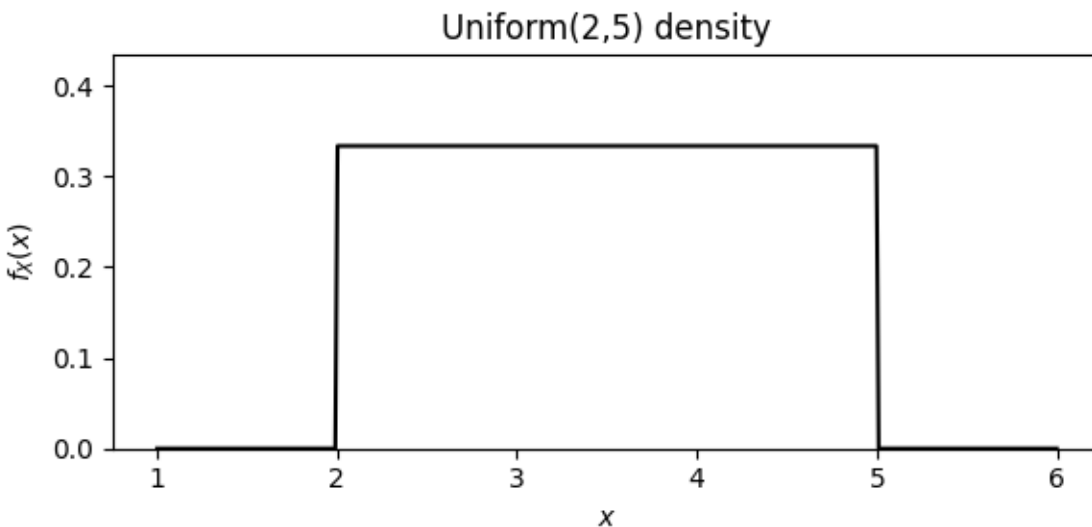
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad

a_u, b_u = 2, 5

def f_uniform(x):
    x = np.asarray(x, dtype=float)
    return np.where((x >= a_u) & (x <= b_u), 1 / (b_u - a_u), 0.0)

xs = np.linspace(a_u - 1, b_u + 1, 400)
fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(xs, f_uniform(xs), color='black')
ax.set_ylim(0, 1 / (b_u - a_u) * 1.3)
ax.set_xlabel('$x$')
ax.set_ylabel('$f_X(x)$')
ax.set_title(f'Uniform({a_u},{b_u}) density')
plt.tight_layout()
plt.show()

total, _ = quad(f_uniform, a_u - 1, b_u + 1)
p_3_4, _ = quad(f_uniform, 3, 4)
print(f"total area:          {total:.4f}")
print(f"P(3 <= X <= 4) =    {p_3_4:.4f}    (should be 1/3 = {1/3:.4f}, since [3,4] is 1/3 of [2,5])")
```



total area: 1.0000
 $P(3 \leq X \leq 4) = 0.3333$ (should be $1/3 = 0.3333$, since $[3,4]$ is $1/3$ of $[2,5]$)

0.3 Example: a PDF can take arbitrarily large values

⚠ A density is *not* a probability

Nothing in the definition requires $f_X(x) \leq 1$ — only that the *total area* equals 1. Squeeze the same unit of probability into a short enough interval and the height of the density can be as large as you like.

i Fact: $P(X = x_0) = 0$ for continuous X

For any single point x_0 ,

$$P(X = x_0) = \int_{x_0}^{x_0} f_X(x) dx = 0,$$

since we're integrating over an interval of zero width. Continuous random variables put *no* probability mass on individual points — only on intervals of positive length.

0.4 Expectation of a continuous random variable

i Definition: Expectation

$$E[X] = \int_{-\infty}^{\infty} x f_X(x) dx.$$

The rules we've seen previously generalize once we replace summations with integrals!

i Theorem: LOTUS for continuous random variables

For any function g ,

$$E[g(X)] = \int_{-\infty}^{\infty} g(x) f_X(x) dx.$$

i Definitions: moments and variance

The k -th **moment** of X is $E[X^k]$. **Variance** is defined exactly as in the discrete case,

$$\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2,$$

and the shortcut formula's proof used only linearity — nothing about summing versus integrating — so it carries over unchanged.

🔥 In-class exercise

Prove that for constants a, b and a continuous random variable X :

$$E[aX + b] = aE[X] + b \quad \text{and} \quad \text{Var}(aX + b) = a^2 \text{Var}(X).$$

Solution: identical to previous discrete proof, with sums replaced by integrals.

$$E[aX + b] = \int (ax + b) f_X(x) dx = a \int x f_X(x) dx + b \int f_X(x) dx = aE[X] + b.$$

$$\text{Var}(aX + b) = E[(aX + b - aE[X] - b)^2] = E[a^2(X - E[X])^2] = a^2 \text{Var}(X).$$

Every fact about expectation and variance built on linearity carries over to continuous RVs with zero extra work.

0.5 The Exponential random variable

The continuous “cousin” of the Geometric — instead of counting *trials* until an event, it measures the *waiting time* until an event that can happen at any instant.

i Definition: Exponential(λ)

$$f_X(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0, \end{cases} \quad \lambda > 0.$$

$$E[X] = \frac{1}{\lambda}, \quad \text{Var}(X) = \frac{1}{\lambda^2}$$

(both derivable by integration by parts) — the same shape of answer as the Geometric(p) mean $1/p$, with λ playing the role of a *continuous rate* in place of p .

Example: customers arrive at a coffee shop at an average rate of $\lambda = 3$ per hour, memorylessly (the time since the last arrival tells you nothing about when the next one comes). The waiting time X until the next customer is Exponential(3), measured in hours, so $E[X] = \frac{1}{3}$ hour = 20 minutes.

```

def f_exp(x, lam):
    x = np.asarray(x, dtype=float)
    return np.where(x >= 0, lam * np.exp(-lam * x), 0.0)

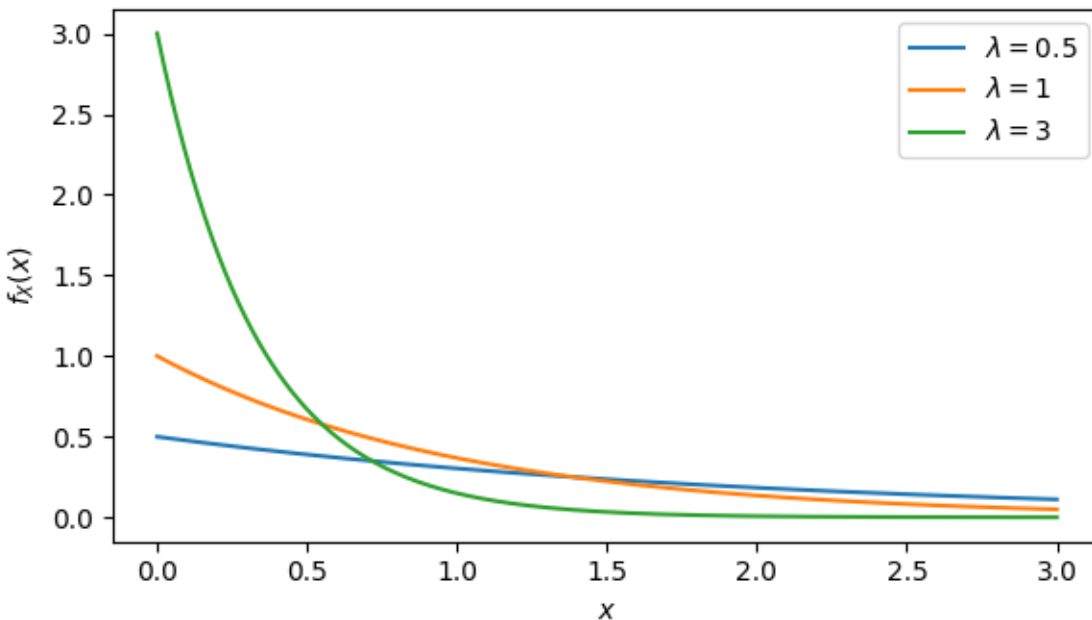
xs = np.linspace(0, 3, 400)
fig, ax = plt.subplots(figsize=(6, 3.5))
for lam in [0.5, 1, 3]:
    ax.plot(xs, f_exp(xs, lam), label=f'$\\lambda={lam}$')
ax.set_xlabel('$x$')
ax.set_ylabel('$f_X(x)$')
ax.legend()
plt.tight_layout()
plt.show()

lam = 3.0 # the coffee shop: 3 customers/hour
E_exp_num, _ = quad(lambda x: x * f_exp(x, lam), 0, 50)
E_exp2_num, _ = quad(lambda x: x ** 2 * f_exp(x, lam), 0, 50)
Var_exp_num = E_exp2_num - E_exp_num ** 2

rng = np.random.default_rng(151)
wait_times = rng.exponential(1 / lam, size=1_000_000) # numpy's scale parameter is 1/lambda

print(f"lambda = {lam}")
print(f"E[X]: formula 1/lambda = {1/lam:.4f} quad = {E_exp_num:.4f} simulated = {wait_times.mean():.4f}")
print(f"Var(X): formula 1/lambda^2 = {1/lam**2:.4f} quad = {Var_exp_num:.4f} simulated = {wait_times.var():.4f}")

```



```

lambda = 3.0
E[X]: formula 1/lambda = 0.3333 quad = 0.3333 simulated = 0.3327
Var(X): formula 1/lambda^2 = 0.1111 quad = 0.1111 simulated = 0.1112

```

0.6 Cumulative Distribution Functions

i Definition: CDF

For *any* random variable X — discrete or continuous —

$$F_X(x) = P(X \leq x).$$

Unlike the PMF or PDF, the CDF is defined the *same way* regardless of type. And each of PMF, PDF, or CDF — whichever applies to X — **unambiguously pins down the entire distribution**: from any one of them, you can recover the others.

0.7 Properties of the CDF

Every CDF, discrete or continuous, satisfies:

- **monotonically nondecreasing**: $x_1 \leq x_2 \implies F_X(x_1) \leq F_X(x_2)$,
- $F_X(x) \rightarrow 0$ as $x \rightarrow -\infty$, and $F_X(x) \rightarrow 1$ as $x \rightarrow \infty$,
- for **discrete** X : F_X is a **piecewise-constant step function** — flat except for a jump of size $p_X(x)$ at every x in the range of X ,
- for **continuous** X : F_X is **continuous** everywhere (no jumps at all — consistent with $P(X = x_0) = 0$ for every x_0).

Relation between PDF and CDF, for X continuous:

$$F_X(x) = \int_{-\infty}^x f_X(t) dt, \quad \text{equivalently} \quad f_X(x) = F'_X(x)$$

wherever F_X is differentiable — the Fundamental Theorem of Calculus!

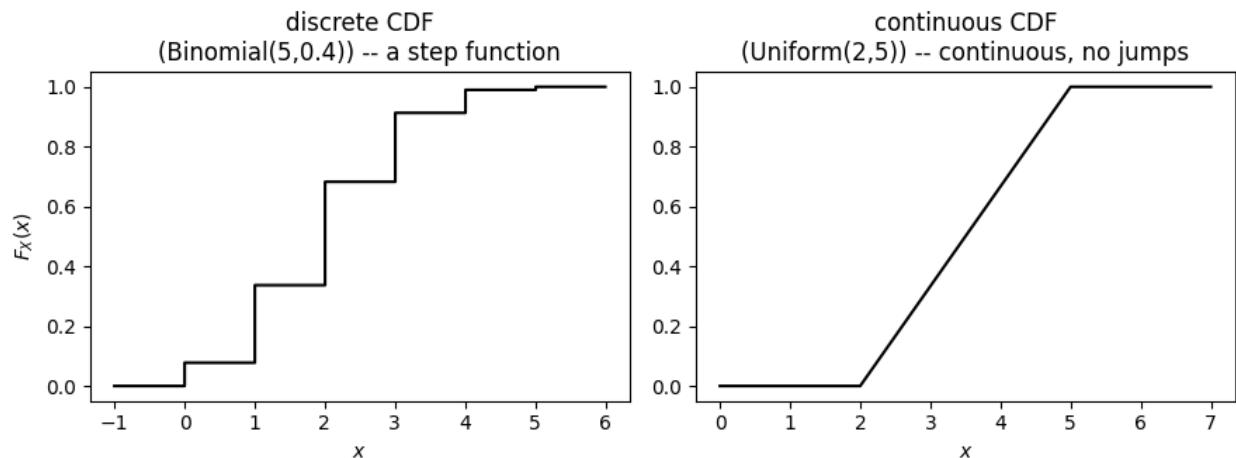
```
from scipy.stats import binom

fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))

# discrete: Binomial(5, 0.4) CDF -- a step function
n_b, p_b = 5, 0.4
xs_d = np.linspace(-1, 6, 700)
cdf_d = binom.cdf(np.floor(xs_d), n_b, p_b)
axes[0].step(xs_d, cdf_d, where='post', color='black')
axes[0].set_title('discrete CDF\n(Binomial(5,0.4)) -- a step function')
axes[0].set_xlabel('$x$')
axes[0].set_ylabel('$F_X(x)$')

# continuous: Uniform(2,5) CDF -- continuous, and (in this special case) piecewise linear
xs_c = np.linspace(0, 7, 400)
cdf_c = np.clip((xs_c - 2) / (5 - 2), 0, 1)
axes[1].plot(xs_c, cdf_c, color='black')
axes[1].set_title('continuous CDF\n(Uniform(2,5)) -- continuous, no jumps')
axes[1].set_xlabel('$x$')

plt.tight_layout()
plt.show()
```



0.8 Geometric and Exponential CDFs

$$X \sim \text{Geometric}(p) : F_X(k) = 1 - (1 - p)^k, \quad k = 1, 2, \dots$$

$$X \sim \text{Exponential}(\lambda) : F_X(x) = 1 - e^{-\lambda x}, \quad x \geq 0.$$

Same shape of formula — because the Exponential really is the **continuous limit** of the Geometric. Chop a unit of time into n tiny slices, let each slice independently “succeed” with probability $p = \lambda/n$, and count *time* (not *trials*) until the first success. As $n \rightarrow \infty$, the Geometric CDF — rescaled so trial number k becomes time k/n — converges to the Exponential(λ) CDF.

💡 Aside

Both distributions are **memoryless** — the property we used to derive the Geometric’s mean carries over exactly to the Exponential, and we’ll put it to work again later this course.

```
lam = 2.0
t = np.linspace(0, 3, 400)
exp_cdf = 1 - np.exp(-lam * t)

fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(t, exp_cdf, 'k--', lw=2, label=f'Exponential( $\lambda={lam}$ ) CDF')

for n in [2, 5, 20, 200]:
    p = lam / n
    k_vals = np.arange(0, int(3 * n) + 1)
    geo_cdf = 1 - (1 - p) ** k_vals
    ax.step(k_vals / n, geo_cdf, where='post', alpha=0.7, label=f'Geometric( $\lambda/n$ ), n={n}')

ax.set_xlabel('time $t = k/n$')
ax.set_ylabel('CDF')
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

# max pointwise CDF gap, shrinking as n grows
n_range = np.unique(np.geomspace(2, 5000, 25).astype(int))
```

```

max_gaps = []
for n in n_range:
    p = lam / n
    k_vals = np.arange(0, int(5 * n) + 1)
    geo_cdf = 1 - (1 - p) ** k_vals
    t_vals = k_vals / n
    exp_at_t = 1 - np.exp(-lam * t_vals)
    max_gaps.append(np.max(np.abs(geo_cdf - exp_at_t)))

```

```

fig2, ax2 = plt.subplots(figsize=(5, 3))
ax2.loglog(n_range, max_gaps, 'o-')
ax2.set_xlabel('$n$')
ax2.set_ylabel('max pointwise CDF gap')
plt.tight_layout()
plt.show()

```

